PYTHON

Module 16: Unique Sets & Frozensets

Understanding Python Sets

Sets are unordered collections used to store multiple items in a single variable. They are ideal for managing unique data points.

Defining Characteristics:

• **Unordered**: Items have no defined sequence and may appear differently each time.

• **Unindexed**: You cannot access items via `myset[0] .`

• **Unique Elements Only**: Duplicates are automatically removed (Note: `True` and `1`, as well as `False` and `0`, are treated as duplicates).

• **Mutable Structure**: While you cannot change specific items, you can add or remove them freely.

Adding & Updating Data

Sets provide specific methods for growth:

- add() : Used to insert a single element into the set.

- update() : Merges another set OR any other iterable (lists, tuples) into the current set.

Benefit: Logic remains consistent. If you update a set with a list that contains duplicates, only the unique values will be merged.

Safe Removal Strategies

Removing data from sets involves two primary methods with a key difference in error handling:

- remove() : Deletes an item; raises a KeyError if the item is not found.
- discard() : Deletes an item; does **not** raise an error if the item is missing.
- pop() : Removes a random element (since sets are unordered, you cannot predict which one).
- clear() and del : Used to empty the set or delete the container variable.

Advanced Set Math (Joining)

Sets in Python mirror mathematical set theory:

- **Union (`union()` or `|`)**: Combines all items from both sets.
- **Intersection (`intersection()` or `&`)**: Keeps only items present in BOTH sets.
- **Difference (`difference()` or `-`)**: Keeps items from the first set that are not in the second.
- **Symmetric Difference (`symmetric_difference()` or `^`)**: Keeps items that are in either set, but NOT in both.

Update variants: Methods like `intersection_update()` modify the original set instead of creating a new one.

The Frozenset: Frozen Logic

A frozenset is an immutable version of a set. Once created, its content cannot be altered.

Why use it? At CodeHive, we use frozensets when we need a set that can act as a key in a dictionary or when we want to ensure data integrity for configuration constants.

Syntax: fs = frozenset(["apple", "banana"])

Functional Note: It supports all joining methods (union, intersection) but none of the update methods.

Set Method Reference

Quick tools for the developer:

- isdisjoint() : Returns True if two sets have no common items.

- issubset() : Checks if all items of set A exist in set B.

- issuperset() : Checks if set A contains all items of set B.

Sets are highly optimized for 'membership testing' (checking if an item exists), performing much faster than lists for large datasets.